

Quantitative Options Coding Agent & Live Trading Analyst - SKILLS & RESPONSIBILITIES.md

Exported: June 07, 2026

Summary

This technical document serves as a comprehensive blueprint for a **quantitative analyst role** dedicated to developing an **automated index-options trading system**. The core objective is to architect a **high-frequency data pipeline** that synchronizes real-time market signals—such as **gamma exposure, volatility skew, and put-call ratios**—to trigger precise buying strategies. Beyond the mathematical modeling of **market microstructure and derivative Greeks**, the text outlines rigorous standards for **low-latency infrastructure** and production-grade software engineering. Ultimately, the source establishes a framework for transforming complex **multi-variable correlations** into a validated, risk-controlled environment for **live intraday trading**.

Keywords: Options Trading Strategy, Quantitative Signal Generation, Real-time Data Pipeline, Feature Engineering, Risk Management Systems

Content

Quantitative Options Coding Agent & Live Trading Analyst - SKILLS & RESPONSIBILITIES
Executive Summary
This role involves designing and implementing an intraday index-options buying strategy using cross-market signals. The analyst will architect a low-latency pipeline to ingest live market and options data, compute synchronized features (PCR regimes, IV skew, dealer gamma exposure, OI concentrations, premium VWAP), and generate composite “GO/NO-GO” signals. Strong emphasis is placed on multi-variable correlations and lead-lag effects rather than isolated indicators. Deliverables include production-ready code modules, real-time monitoring dashboards, backtest results, and thorough documentation.
Responsibilities & Skill Mapping
Data Pipeline Development: Build

real-time ingestion of market quotes, trades, option chains (tick/L2 feeds, OPRA, exchanges). Ensure normalization, time synchronization, and storage (e.g. kdb+, Kafka, or Redis) for live features.

Feature Engineering: Implement calculations for PCR regimes, IV skew, theta decay acceleration, net gamma exposure, OI walls, Max Pain, and premium/spot VWAP. Use efficient numerical libraries to update these per time bucket.

Signal Generation: Code the composite algorithms (Momentum Index, Vol-Crush Trap detector, Gamma-Hedge triggers, No-Trade filters) using synchronized features. Tune thresholds and logic for PCR/IV regime detection.

Backtesting & Analysis: Design walk-forward backtests on historical tick/option data. Quantify strategy performance (Sharpe, drawdown, hit-rate) including realistic slippage and transaction costs. Iterate on model refinements.

Deployment & Monitoring: Deploy code to low-latency servers (colocated exchanges). Build dashboards/alerts (Grafana/Plotly) for key indicators (PCR, GEX, Max Pain shifts). Implement risk checks (position/duress limits) and kill-switch on strategy breaches.

Documentation & Code Quality: Write clean, modular code with unit tests. Maintain version-controlled documentation on data definitions, algorithm logic, and test cases. Ensure reproducibility (Docker/Conda environments) and peer review compliance.

Technical Skills & Technology Stack

Languages: Python (primary for analytics), C++/Rust/Java (low-latency services), SQL (time-series DBs), Bash for scripting. Intermediate Scala/Go (if using specific data tools).

Numerical Libraries: NumPy, Pandas, SciPy, TA-Lib, cuDF (GPU), TensorFlow/PyTorch (for any ML calibration), QuantLib (option models).

Data Tools: kdb+/q or TickDB for historical tick storage, Kafka/Apache Pulsar or ZeroMQ for streaming, Redis/memcached for in-memory state. Use OPRA/OPTx feeds or vendor APIs (QuantHouse, USTP) for options.

Analytics: Jupyter, Matplotlib/Seaborn for plotting, Pandas/HDF5 or Parquet for time-series. SQL or Dask for batch aggregation.

Infrastructure: Linux, Docker/Kubernetes for deployment, CI/CD (Jenkins/GitHub Actions) for automated testing, AWS/GCP for cloud batch jobs if needed.

Risk & Execution: FIX engines (QuickFIX/J), low-latency sockets, order management systems. Quant-focused DBs (Resque, KDB) and monitoring (Grafana, InfluxDB).

Versioning & QA: Git, GitLab; pytest/unittest for code; documentation tools (Sphinx, Markdown); CI pipelines.

Quantitative Skills

Statistical Analysis: Regression, cross-correlation, covariance, hypothesis testing to identify leading/lag relationships between features.

Time-Series Modeling: Autoregressive models, EWMA/VWAP filters, Fourier/wavelet (for intraday seasonality), and feature smoothing.

Signal Processing: Lead-lag detection via cross-correlograms and Granger causality on tick-level data.

Options Theory: Derivation of option Greeks (Δ , Γ , Θ , Vega). Modeling implied volatility surfaces; familiarity with Black-Scholes and SABR fits.

Gamma Exposure Modeling: Summation of gamma * open interest to estimate net dealer hedges. Detect "gamma flip" levels where net GEX changes sign.

Optimization & ML (if needed): Parameter calibration (e.g. PCR threshold), possibly reinforcement learning for execution; risk model (VaR, CVaR).

Microstructure Knowledge: Tick-level flow,

order book dynamics, slippage estimation, and effects of trade size on VWAP.

Data Sources & Formats

Market Data: Live tick (trades/quotes) and L2 orderbook feeds (e.g. Cboe, CME MDP). Historical tick data (Nanex, Exegy, or Tick Data, Inc.).

Options Data: Streaming OPRA quotes (bid/ask, volume, open interest) for all strikes/ expirations of the target index (e.g. SPX). Historical options chains with daily OI snapshots.

Data Vendors: Bloomberg Terminal (B-PIPE), Refinitiv, or specialized vendors like QuantHouse for low-latency feeds.

Formats: Binary time-series (kdb+/q tables), Parquet/CSV for batch archives. OPRA messages or FIX data for live ticks.

Reference Data: Corporate actions, holiday calendars, etc. Ensure sync clocks (NTP) for timestamp alignment.

Feature Engineering (Expanded Parameter Matrix)

Develop code to compute the following per time interval (e.g. 1min or tick-aligned buckets), always synchronizing all inputs:

Put/Call Ratio (PCR): **Formula:** $PCR = (\text{option volume or OI, choose consistent definition})$. **Regimes:** Define baseline ~ 0.7 (neutral). $PCR < 0.7 = \text{Bullish regime}$. $PCR 0.7-0.8 = \text{Neutral regime}$. $PCR > 1.1 = \text{Bearish regime}$. **Trend:** Compute slope $PCR_{slope} = d(PCR)/dt$ over short windows.

Implied Volatility (IV) Skew: **Compute:** Fit an IV surface or directly calculate difference between call and put IVs across strikes. Or use slope of IV vs. strike. Detect expansion vs. contraction: $\Delta IV_{skew}/dt$. **Sample pseudocode:** $iv_{skew} = iv_{surface}[strike_call_ATM] - iv_{surface}[strike_put_ATM]$.

Theta Acceleration: **Idea:** Compute time-decay rate change of near-ATM options. **Compute:** Estimate Θ (change of price per day), then track $d^2\Theta/dt^2$ over short intervals. **Logic:** Rapidly increasing negative theta indicates accelerating time-decay (bad for longs), e.g., $theta_accel = (Theta_new - Theta_old) / \Delta t$.

Gamma Exposure (GEX): **Aggregate:** For each strike: $gamma_OI = option_gamma(strike) * OI(strike)$. Sum over calls/puts (use spot-price scaling if needed). **Net GEX:** $net_gamma = \sum (\Gamma_{call} OI_{call}) - \sum (\Gamma_{put} OI_{put})$. **Interpretation:** Positive net GEX (dealers long Γ) implies market makers **sell into rallies** (dampening moves); negative net GEX implies **buy on rallies** (amplifying moves). **Thresholds:** Identify “zero-Gamma flip zones” where $net_gamma \approx 0$.

OI Walls & Smart Money Positions: **Total OI Walls:** Identify large cumulative open interest at key strikes (calls vs puts). **Intraday OI Change:** Track ΔOI per strike since open. Rapid additions on CE/PE indicate institutional straddle/strangle (premium selling) or unwinds (covering). **Smart Straddle/Strangle:** Compute net changes: e.g., Large simultaneous OI increases on adjacent CE and PE = dealer writing.

Max Pain & Shifts: **Definition:** Strike minimizing combined payoff to option buyers. **Compute:** For each strike K , calculate payoff sum if underlying closes at K : then $MaxPain = \text{argmin}_K P\&L(K)$. **Track:** Monitor shifts in MaxPain level intraday. Stationary MaxPain suggests balanced positioning; moving MaxPain indicates directional bias from dealers.

Premium VWAP vs Spot VWAP: **Premium VWAP:** Compute VWAP of NTM (near-the-money) option premiums. **Spot VWAP:** Standard VWAP of underlying (see Figure below). **Comparison:** When premium-VWAP diverges from spot-VWAP (normalized), it signals relative flow into option

premiums vs. spot. A rising premium-VWAP ahead of spot may indicate accumulation of calls. Figure: Example intraday price bars (black) with Volume-Weighted Average Price (VWAP, blue) overlay. VWAP forms a volume-weighted “center-of-gravity” for price. In practice, we compute premium-VWAP for ATM/Near-ATM options and compare it to spot-VWAP to identify when options are relatively expensive or cheap. Figure: Implied Volatility Skew example – call IV (cyan), put IV (purple), and smoothed combined skew (orange) across strikes. This “smile” illustrates higher OTM volatility. In models, we track steepening vs. flattening of this skew curve. Composite Signal Algorithms Design algorithms that combine the above features. Example pseudocode logic: Vol-Crush Bull-Trap Detector:

When bullish PCR (1.1–1.4) coincides with **narrowing** IV skew (call and put IV converging) and accelerating negative theta, signal a likely **volatility crush**. This avoids buying into a false breakout when bullish sentiment is overheated but implied volatility is collapsing. Momentum Breakout Signal:

Look for PCR in neutral/bullish band (e.g. 0.7–1.1), coupled with expanding IV (skew widening), surging call OI, and premium-VWAP rising above spot-VWAP. This composite suggests genuine institutional buying and fast move. Gamma Hedging Trigger:

When spot price approaches a “zero-Gamma flip” zone under heavy OI walls, anticipate market maker hedging to move price. For example: Dealers short gamma will create momentum once the underlying drifts beyond their neutral zone. Institutional Accumulation/Distribution:

Identify divergence between spot and option flows: e.g., if price is range-bound but NTM call premium VWAP breaks out above its VWAP and PCR is climbing while ATM call OI falls (short covering), signal a hidden accumulation. No-Trade / Trap Signature: Flat PCR (~neutral), stationary MaxPain, and large simultaneous OI addition on adjacent CE and PE (dealer strangle writing) imply premiums are being sold into. Formalize as: Mathematical Logic Summary PCR Regime Index:

= .

Classify into regimes (e.g., PCR < 0.7 bullish, > 1.1 bearish). IV Skew Slope:

= $IV_{\{\}}() - IV_{\{\}}()$,

track derivative $d()/dt$ for crush vs. expansion. Theta Acceleration:

= -, = . Gamma Exposure (GEX):

$_{\{\}}(K) OI(K)$.

Net across strikes indicates overall dealer hedge bias. Max Pain:

= $_K \cdot VWAP$:

= ,

applied separately to underlying and to option premium flows. Testing & Backtesting Datasets: Use multi-year historical data (e.g. SPX index and SPX options from 2016–2025). Include tick-level prices, time-stamped options trades, and end-of-day/real-time OI. If possible, include ODTE data. Backtest Metrics: Evaluate returns,

Sharpe ratio, win-rate, maximum drawdown. Stress-test under volatile vs calm regimes. Use walk-forward cross-validation to simulate deployment. Costs & Slippage: Model realistic slippage (e.g. 1-2 ticks per leg) and commission (e.g. \$1-\$2 per contract). Incorporate bid-ask spreads into P&L. Benchmarking: Compare to naive strategies (e.g. always buy ATM calls at open). Ensure our "edge" (excess return) is statistically significant. Regression & Robustness: Test sensitivity of signal thresholds. Ensure small parameter shifts don't grossly flip results. Maintain unit and integration tests (pytest) for code correctness. Deployment & Runbook Real-Time Pipeline (Mermaid): Design a multistage streaming pipeline. Market data feeds are ingested via a low-latency gateway, normalized, and passed to the feature computation engine. The engine updates indicators in real time, publishing signals to the execution module. The order router applies risk checks before sending orders to exchanges. A parallel dashboard process visualizes key metrics and triggers alerts. Latency Budgets: Keep pipeline latency < 5-10ms for entire ingest-to-signal path. Co-locate servers near exchange gateways. Use multithreading and lock-free queues. Risk Controls: Hard stop-loss and maximum position limits. For example, if realized loss > 1% portfolio or single-day drawdown > 0.5%, auto-disable signals. Kill-switch procedures for systemic failures (e.g. network loss). Monitoring: Continuously track data feeds health, model outputs, and P&L. Generate alerts on anomalies (e.g. feature NaN, backtest drift). Use Grafana/Prometheus or equivalent. Model Refresh: Update models and parameters (PCR baseline, thresholds) at regular intervals (weekly or monthly) based on new data. Recalibrate IV surface fittings daily (intraday if computationally feasible). Time-of-Day Structural Filters **Table:** Interpretation of PCR trends, Max Pain shifts, and OI changes across intraday segments. Assumptions & Constraints Markets/Assets: Focus on a major index (e.g. SPX) or its ETF (e.g. SPY). Data from Cboe/CME for SPX options and corresponding futures/spot. Capital: Strategy sized for institutional scale (e.g. 20M-100M notional). Position limits set accordingly. Latency: Target sub-10ms round-trip from tick arrival to order entry; assume colocated hardware or premium cloud networking. Data Vendors: Use OPRA-level data (e.g. directly from exchanges or via a low-latency vendor like OCC or QuantHouse). License end-of-day OI from OCC or FRAC. Execution Environment: Linux servers with high-performance NICs; assume 24/5 market hours (US markets). Risk Limits: E.g., max 50% of capital in any single direction, stop-trades if intraday loss > 2%. Regulations: Comply with relevant exchange rules (e.g. minimum tick sizes), no-usury of insider information. Deliverables & Acceptance Criteria Code Repositories: Well-structured Git repos for data ingestion, feature engines, signal logic, and execution. All code must compile/build without errors. Configuration: Deployment scripts (Dockerfiles, Kubernetes manifests) for production rollout. Tests: Unit tests covering edge cases for feature calculations; integration tests for pipeline end-to-end (with synthetic feeds). Backtest Results: Detailed report with P&L charts, risk metrics, strategy

description. Tables comparing test vs baseline. Dashboards: Interactive visualizations (e.g. Real-time PCR, GEX, MaxPain levels) and example screenshots. Documentation: This SKILLS.md plus detailed README/Confluence for setup. Include API docs (Docstrings/Sphinx) and data dictionary. Success is measured by: all tests passing, documentation completeness, and a convincing backtested edge (performance metrics meeting targets) approved by stakeholders. Documentation & Standards Follow Markdown for text; PEP8 for Python code. Comment complex logic extensively. Use version control (git) with code reviews. Write unit tests for all formulas and functions. Provide notebooks or scripts to reproduce key analyses (data sync, signal thresholds). Maintain change logs for models and parameter updates. Ensure reproducibility: share seed values, data snapshots, and environment specs. Use continuous integration to enforce quality.